



**Faculty: BSC(Hons) Computer System Engineering**

**NAME: Binod Katuwal**

**Student Id: 250825048**

**MODULE CODE: CET138**

**MODULE TITLE: Full Stack Development**

**MODULE LEADER: David Grey**

**SUBMISSION DATE AND TIME: Fri, 19 Jun 2026**

**ASSIGNMENT: 1**

**Academic Misconduct is an offence under university regulations, and this involves:**

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

**STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:**

|                                                                                    |            |
|------------------------------------------------------------------------------------|------------|
| • I have used artificial intelligence tools to generate an idea for my assignment: | <b>YES</b> |
| • I have used artificial intelligence tools to write my assignment for me:         | <b>NO</b>  |
| • I have used artificial intelligence tools to brainstorm ideas for my assignment: | <b>NO</b>  |
| • I have used artificial intelligence tools to correct my original assignment:     | <b>NO</b>  |

**DECLARATION**

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.
- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

CET138 Full Stack Development

# FULL STACK DEVELOPMENT

| Details         |                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| Name            | Binod Katuwal                                                                                                 |
| Student Id      | 250825048                                                                                                     |
| Module Code     | CET138                                                                                                        |
| Module Title    | Full Stack Development                                                                                        |
| Module Leader   | David Grey                                                                                                    |
| Submission Date | Fri, 19 Jun 2026                                                                                              |
| Assignment      | Assignment 1 – portfolio (30%)                                                                                |
| Project         | Forever Moments – Wedding Planner Website                                                                     |
| Live Demo       | <a href="https://binodkatuwal.github.io/Forever-Moments/">https://binodkatuwal.github.io/Forever-Moments/</a> |

## Contents

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| 1.  | Introduction to Full Stack Development.....                | 4  |
| 2.  | The Three Layers of a Web Application .....                | 4  |
| 2.1 | The Presentation Layer (Front End) .....                   | 4  |
| 2.2 | The Application Layer (Back End) .....                     | 5  |
| 2.2 | The Data Layer (Database) .....                            | 5  |
| 3.  | Architecture Overview .....                                | 6  |
| 4.  | Popular Full Stack Combinations .....                      | 6  |
| 5.  | Front End vs. Back End vs. Full Stack Roles .....          | 7  |
| 5.1 | Front End Developer .....                                  | 7  |
| 5.2 | Back End Developer .....                                   | 7  |
| 5.3 | Full Stack Developer.....                                  | 8  |
| 6.  | The Request-Response Cycle .....                           | 8  |
|     | Step-by-Step Breakdown .....                               | 8  |
| 7.  | APIs and How Front End and Back End .....                  | 9  |
| 8.  | Deployment and DevOps Awareness .....                      | 10 |
| 9.  | Full Stack Development in the Context of This Module ..... | 10 |
| 10. | Career Relevance and Industry Demand.....                  | 11 |
| 11. | Summary .....                                              | 12 |
|     | For Full Page Review Link .....                            | 12 |

# What is Full Stack Development?

## 1. Introduction to Full Stack Development

Full stack development refers to the practice of working across all layers of web application – from the user interface that a person interacts with, through the logic that processes data, and down the database that stores information. A full stack developer is someone who has the knowledge and skills to work on both the front end (client-side) and back end (server-side) of a web application, as well as understands how these layers communicate with one another.

The term “stack” refers to the collection of technologies – programming language, frameworks, libraries, database and tools – that are stacked together to build a complete, functional web application. When a developer is described as “full stack”, it means they are competent across the entire stack rather than specializing exclusively in one layer.

In modern software development, full stack developers are highly valued because they can contribute to all parts of a project, understand how each component interacts, and reduce the communication overhead that might arise between separate front-end and back-end specialists.

## 2. The Three Layers of a Web Application

A typical web application is structured across three main layers, often referred to as the three-tier architecture. Understanding how these layers work together is fundamental to full stack development.

### 2.1 The Presentation Layer (Front End)

The presentation layer is everything the user sees and interacts with directly in their web browser. It is responsible for the visual layout, styling, user interface components, and client-side interactivity. The key technologies used in this layer are:

- HTML (Hyper Text Markup Language) – defines the structure and content of web pages.
- CSS (Cascading Style Sheets) – controls the visual appearance, layout, and responsiveness.

- JavaScript – adds interactivity, handles user events, and dynamically updates content
- Frameworks and libraries such as React, Vue.js, and Angular – help build complex user interfaces more efficiently.
- Bootstrap – a CSS framework providing per-built responsive components

The front end communicates with the back by sending HTTP requests (Typically via AJAX or the Fetch API) and receiving responses, usually in JSON format.

## 2.2 The Application Layer (Back End)

The application layer – also called the server side – contains the business logic of the application. It processes incoming requests from the front end, applies rules and logic, communicates with the database, and returns the appropriate response. Technologies commonly used which mention below :

- Server-side language: Node.js (JavaScript), Python (with Flask or Django), PHP, java, Ruby
- Web frameworks: Express.js, Spring Boot, Laravel, Ruby on Rails
- RESTful APIs and GraphQL – define how the front end and back end exchange data
- Authentication and authorization – managing user sessions, tokens (JWT), and access control

The black end sits between the front end and the database, ensuring data is retrieved, processed, and returned securely and efficiently.

## 2.2 The Data Layer (Database)

The data layer is responsible for storing, retrieving, and managing the application's data. It is managed by a Database Management System (DBMS). There are two main categories:

- Relational database (SQL): MySQL, PostgreSQL, SQLite – store unstructured data in tables with defined relationships
- Non-relational database (NoSQL): MongoDB, Firebase, Redis – store unstructured or semi-structured data in formats such as JSON documents or key-value pairs

The back end communicates with the database using queries (e.g. SQL SELECT statements) or an Object Relational Mapper (ORM) such as sequelize or Mongoose, which abstracts the database queries into code.

### 3. Architecture Overview

The following diagram illustrates how the three layers interact in a typical full stack web application:

| Layer              | Technology Examples                     | Responsibility                          |
|--------------------|-----------------------------------------|-----------------------------------------|
| Front End (Client) | HTML, CSS, JavaScript, React, Bootstrap | Display, user interaction, UI rendering |
| Back End (Server)  | Node.js, Express, Python/Django, PHP    | Business logic, routing, API handling   |
| Database           | MySQL, MongoDB, PostgreSQL, Firebase    | Data storage, retrieval, persistence    |

When a user interacts with a web page – for example, submitting a login form – the following sequence of events occurs across all three layers.

- The user enters their credentials and clicks Submit (Front End)
- JavaScript captures the form data and sends an HTTP POST request to server
- The database returns the user record if the credentials are correct
- The back end generated a session token or JWT and responds with a success message
- The front end receives the response and redirects the user to their dashboard

### 4. Popular Full Stack Combinations

Different combinations of technologies can be used to build a full stack application. These combinations are often referred to by acronyms. Below are some of the most widely used stacks in the industry today:

| Stack Name     | Technologies                                | Best Known For                                         |
|----------------|---------------------------------------------|--------------------------------------------------------|
| MERN           | MongoDB, Express.js, React, Node.js         | Modern single-page applications, JavaScript throughout |
| MEAN           | MongoDB, Express.js, Angular, Node.js       | Enterprise apps, strong TypeScript support             |
| LAMP           | Linux, Apache, MySQL, PHP                   | Traditional web hosting, WordPress, CMS platforms      |
| MEVN           | MongoDB, Express.js, Vue.js, Node.js        | Lightweight SPAs, progressive web applications         |
| Django + React | Python/Django (back end), React (front end) | Data-heavy applications, rapid development             |

In this module, we focus on core full stack technologies: HTML, CSS, Bootstrap, JavaScript for the front end, with the understanding that back-end and database skills build on top of this foundation.

## 5. Front End vs. Back End vs. Full Stack Roles

It is important to understand the distinction between the different developer specialization in the industry:

### 5.1 Front End Developer

A front-end developer focuses exclusively on the presentation layer. They work with HTML, CSS, and JavaScript to build user interfaces that are visually appealing, accessible, and responsive across different devices. They are often closely involved with UX/UI design decisions.

### 5.2 Back End Developer

A back-end developer works on the server-side logic, databases, and APIs. They handle data processing, authentication, server configuration, and ensure the application performs efficiently and securely. They may work with languages such as Python, Java, PHP, or Node.js.

## 5.3 Full Stack Developer

A full stack developer can work across both the front end and back end. They understand the full request-response cycle, can build and consume APIs, configure databases, and deploy applications. While they may not have the depth of a specialist in either area, their breadth of knowledge makes them extremely versatile — especially valuable in small teams or startups.

| Role           | Primary Skills                   | Typical Responsibilities                                      |
|----------------|----------------------------------|---------------------------------------------------------------|
| Front End Dev  | HTML, CSS, JS, React/Vue/Angular | UI design, accessibility, browser compatibility               |
| Back End Dev   | Node.js, Python, Databases, APIs | Server logic, security, performance, data management          |
| Full Stack Dev | All of the above                 | Feature Development across all layers, architecture decisions |

## 6. The Request-Response Cycle

One of the most fundamental concepts in full stack development is understanding the HTTP request-response cycle, which governs how a browser (client) and a web server communicate.

### Step-by-Step Breakdown

Step 1: The user opens a browser and types a URL (e.g. <https://Google.com/search>)

Step 2: The browser sends an HTTP GET request to the server at that URL

Step 3: The server (back end) receives the request and processes it

Step 4: The back end queries the database for product data

Step 5: The database returns the requested records to the back end

Step 6: The back end packages the data (Often as JSON) and send an HTTP response

Step 7: The browser (front end) receives the response and renders the data as HTML



HTTP methods define the type of action being requested. The most common are:

- GET – retrieve data (e.g. fetch a list of products)
- POST – send data to create a new resource (e.g. submit a form)
- PUT / PATCH – update an existing resource
- DELETE – remove a resource

HTTP status code indicates the outcome of a request. Common examples include 200 OK (success), 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, and 500 internal Server Error.

## 7. APIs and How Front End and Back End

An API (Application Programming Interface) is the contract between the front end and back end. Rather than the server sending entire HTML pages, modern web applications use APIs to exchange structure data – typically in JSON format – allowing the front end to dynamically update the page without a full reload.

A RESTFUL API (Representational State Transfer) is the most common architectural style. It uses HTTP methods and standard URL patterns to define operations on resources. For Examples:

| HTTP Method | URL Endpoint | Action                  |
|-------------|--------------|-------------------------|
| GET         | /api/users   | Retrieve all users      |
| GET         | /api/users/1 | Retrieve user with ID 1 |
| POST        | /api/users   | Create a new user       |
| PUT         | /api/users/1 | Update user ID 1        |
| DELETE      | /api/users/1 | Delete user with ID 1   |

The front end uses the JavaScript Fetch API or libraries such as Axios to make these requests asynchronously, meaning the page remains interactive while data is being loaded in the background.

## 8. Deployment and DevOps Awareness

A full stack developer is also expected to have a basic understanding of how applications are deployed and maintained in production environments. The things included here is shown below:

- Version control: Git and GitHub are used to track changes and collaborate on code
- Hosting platforms: Applications can be deployed to services such as Netlify, Vercel (front end), Heroku, Railway, or AWS (Back end and Full stack)
- Environment variables: Sensitive configuration data such as database passwords and API keys are stored in .env files and never committed to version control
- Build tools: Tools such as Webpack, Vite, and npm scripts are used to bundle, minify, and optimize front-end assets for production
- CI/CD (Continuous Integration / Continuous Deployment): Automated pipelines that test and deploy code whenever a new commit is pushed

While deep DevOps knowledge is typically the responsibility of a specialist, full stack developers working in small teams or on personal project are expected to be comfortable with these concepts.

## 9. Full Stack Development in the Context of This Module

This module (CET 138 Full Stack Development) covers the core technologies used on the client side of full stack development. The skills developed are directly applicable to roles a full stack, front-end, or web development:

| Module Topic        | Role in Full Stack                                                            | Layer                         |
|---------------------|-------------------------------------------------------------------------------|-------------------------------|
| HTML                | Defines the structure and semantic content of web pages                       | Front End                     |
| CSS                 | Controls visual presentation and responsive layouts                           | Front End                     |
| Bootstrap Framework | Rapid UI development with pre-built responsive components                     | Front End                     |
| JavaScript          | Adds interactivity and dynamic behavior; foundation for back-end JS (Node.js) | Front End / Full Stack Bridge |

Mastery of these front-end technologies is the essential first step toward becoming a full stack developer. JavaScript in particular acts as the bridge between front end and back end – with Node.js enabling server-side JavaScript, meaning a developer who knows JavaScript well can theoretically work across the entire MERN or MEAN stack.

## 10. Career Relevance and Industry Demand

Full stack development is one of the most in demand skill sets in the technology industry. According to industry surveys such as the Stack Overflow Developer Survey, full stack developer is consistently among the most common job titles reported by professional developers worldwide.

Some of the key points reasons why full stack skills are highly sought after include:

- **Versatility:** Full stack developers can contribute to any part of a project, making them extremely adaptable
- **Startup suitability:** Small teams often need one person to handle both front-end and back-end tasks
- **Cost efficiency:** Hiring one full stack developer can be more cost-effective than hiring two specialists
- **Project ownership:** Full stack developers can build and ship complete features independently

- Career progression: Full stack knowledge provides a strong foundation to specialize later in areas such as cloud architecture, DevOps, or mobile development

For junior developers graduating from programs like this one, demonstrating full stack capability – even at a foundational level – is a significant advantage when applying for roles.

## 11. Summary

Full stack development is the practice of building complete web applications by working across all layers: the front end (presentation), back end (application logic), and database (data storage). A full stack developer understands how these layers interact through the HTTP request-response cycle and APIs.

The key concepts covered in this section are:

- The three-tier architecture: presentation, application, and data layers
- Common full stack technology combinations (MERN, MEAN, LAMP, etc.)
- The distinction between front-end, back-end, and full stack developer roles
- How HTTP methods and status codes govern client-server communication
- The role of RESTful APIs in connecting front end and back end
- Basic deployment and DevOps concepts relevant to full stack development
- How the topics in this module (HTML, CSS, Bootstrap, JavaScript) fit into the full stack picture

## For Full Page Review Link

Here is the web page link host using GitHub which link is provided below:

<https://binodkatuwal.github.io/Forever-Moments/>